[SESSION 07] $ ./SQL

آنلاین •

رایگان

دورهٔ رایگان مهندسی داده و هوش مصنوعی

جلسهٔ اول

SQL، قسمت اول:
از فایل تا کوئری

دیتابیس چیه و SQL چرا اومد، خوندن و ترکیب داده، فکر کردن با مجموعه‌ها و تله‌های رایج.

تاریخ

سه‌شنبه ۵ آبان

ساعت

۲۰:۰۰



ارائه‌دهنده

کوثر اصغری

مدرس دوره

t.me/data_ai_engineering

من کی‌ام؟

EDUCATION

تحصیلات

فارغ‌التحصیل کارشناسی مهندسی هوافضا دانشگاه صنعتی شریف

EXPERIENCE

تجربه

سرپرست تیم مدیریت داده در حوزه کارگزاری بین‌المللی

مهندس تحلیل داده و مشاور داده کیمیاگران نفت و LABELL

تحلیلگر داده و محصول یکتانت، پرس‌لاین، Nexu و eVTOL.ae

امروز چی می‌گیم؟

01 · 10'

داده قبل از SQL

فایل‌ها، دردسرهاشون و اینکه دیتابیس چرا به وجود اومد

02 · 13'

الفبای دیتابیس

جدول، کلید و خانواده‌ی دستورها با مثال‌های کاربردی

03 · 15'

خواندن داده

SELECT، WHERE، GROUP BY و ترتیب اجرای کوئری

04 · 10'

ترکیب جدول‌ها

انواع JOIN، UNION، EXISTS و CTE

05 · 5'

حلقه نزن!

فکر کردن با مجموعه‌ها و آشنایی با Window Function

06 · 22'

شش تله و جمع‌بندی

اشتباه‌های رایجی که خطا نمی‌دن ولی جواب غلط می‌دن

[SETUP] $ docker compose up -d

محیط تمرین رو بالا بیارید

STEP 1

دریافت

پوشه‌ی sql-course رو از کانال دوره بگیرید و داخلش برید. همه‌ی تمرین‌های هر دو جلسه روی همین داده‌ست.

```
cd sql-course
```

STEP 2

بالا آوردن

داکر دیتابیس PostgreSQL رو می‌سازه و داده‌ی یک فروشگاه آنلاین رو خودش می‌ریزه. بار اول حدود یک دقیقه طول می‌کشه.

```
docker compose up -d
```

STEP 3

اتصال

از مرورگر با Adminer وصل بشید، یا با هر ابزاری مثل DBeaver و psql. کاربر و رمز: student

```
localhost:8080
```

بخش ۱۰

داده قبل از SQL
کجا زندگی می‌کرد؟

از فایل‌های معمولی تا دیتابیس، و اینکه SQL چرا به وجود اومد.

اول همه‌چیز فایل بود

```
# orders.csv
order_id,customer,amount,date
1001,Sara,250,2026-09-01
1002,Ali,120,2026-09-01
1003,Sara,90,2026-09-02

# how much did Sara buy?
total = 0
for line in open("orders.csv"):
    cols = line.strip().split(",")
    if cols[1] == "Sara":
        total += int(cols[2])
```

BEFORE

داده توی یک فایل متنی بود؛ هر خط یک سفارش.

QUESTION

«سارا چقدر خرید کرده؟» برای همین سوال ساده باید کد بنویسیم.

HOW

باز کن، خط به خط بخون، جدا کن، جمع بزن؛ همه با خودمون.

NEXT

سوال بعدی؟ دوباره یک کد تازه از اول.

دردسرهای کار با فایل

READ

خوندن و جست‌وجو

برای هر سوال باید کل فایل رو از اول بخونیم؛ یک میلیون خط یعنی گند.

WRITE

نوشتن همزمان

دو برنامه با هم بنویسن، داده خراب یا گم می‌شه.

CRASH

خرابی وسط کار

برق بره وسط نوشتن، نصف سفارش ثبت شده و نصفش نه.

QUALITY

کیفیت داده

هیچ‌کس جلوی مبلغ منفی یا ایمیل خالی رو نمی‌گیره.

RELATIONS

ربط بین فایل‌ها

مشتری‌ها یک فایل، سفارش‌ها یک فایل؛ وصل کردنشون با خودمونه.

ACCESS

دسترسی

یا کل فایل رو داری یا هیچی؛ نمی‌شه گفت فلانی فقط بخونه.

نکتهٔ کلیدی

دیتابیس هم داده رو روی فایل نگه می‌داره؛ فقط دردسرهاش رو خودش حل می‌کنه.

سیستم مدیریت دیتابیس (DBMS) خوندن، نوشتن، کار همزمان، بازیابی بعد از خرابی و دسترسی رو مدیریت می‌کنه. ما فقط می‌گیم چی می‌خوایم.

یک کوئری از کجا تا کجا می‌ره؟



SQL زبان «اعلانی»ه: «چی» رو می‌گی، دیتابیس خودش «چطور» رو پیدا می‌کنه.

```
SELECT SUM(amount) FROM orders WHERE customer = 'Sara';
```

همون کار دوازده خط پایتون، در یک خط؛ و برای سوال بعدی فقط یک خط عوض می‌شه.

SQL از کجا اومد؟

1970

ایده‌ی جدول

ادگار کاد در IBM پیشنهاد داد داده رو به شکل جدول ببینیم و با منطق ریاضی ازش سوال بپرسیم.

Edgar F. Codd

1974

زبان SEQUEL

دو نفر در IBM زبانی ساختن که شبیه جمله‌ی انگلیسی بود؛ بعداً اسمش شد SQL.

IBM System R

1986

استاندارد شد

SQL استاندارد ANSI و بعد ISO شد؛ از اون موقع دیتابیس‌ها یک زبان مشترک دارن.

ANSI · ISO

TODAY

هنوز همه‌جاست

PostgreSQL، MySQL، ClickHouse، BigQuery و حتی Spark همه SQL می‌فهمن.

50+ years

بخش ۰۲

الفبای
دیتابیس رابطه‌ای

جدول، ردیف، ستون، کلید و خانواده‌ی دستورها.

```
$ SELECT * FROM customers LIMIT 4;
```

جدول، ردیف، ستون

customer_id	full_name	city	country	signup_at
1	Omid Moradi	NULL	Canada	2024-11-23 08:51
2	Fatemeh Moradi	Tabriz	Iran	2026-08-04 22:47
3	Babak Ghasemi	Tehran	Iran	2026-05-20 06:00
4	Sara Ghasemi	Shiraz	Iran	2025-06-17 14:00

جدول یعنی یک موضوع؛ اینجا «مشتری‌ها».

ستون یعنی یک ویژگی (شهر) و ردیف یعنی یک مورد (یک مشتری). customer_id کلید اصلیه: هیچ‌وقت تکرار نمی‌شه.

NULL یعنی «نمی‌دونیم»؛ نه صفره، نه متن خالی. شهر مشتری شماره‌ی ۱ رو نداریم.

جدول‌ها با کلید به هم وصل می‌شن

CUSTOMERS

هر مشتری یک بار

اسم و شهر مشتری فقط همین‌جا ذخیره می‌شه.

```
customer_id | full_name    | city
----------- | ------------ | ------
4           | Sara Ghasemi | Shiraz
```

PRIMARY KEY (customer_id)

ORDERS

هر سفارش فقط شماره‌ی مشتری

سفارش اسم و شهر رو تکرار نمی‌کنه؛ فقط می‌گه مال کدوم مشتریه.

```
order_id | customer_id | amount
-------- | ----------- | ------
1001     | 4           | 250
1003     | 4           | 90
```

FOREIGN KEY (customer_id) → customers

خانواده‌ی دستورها

خانواده	یعنی چی؟	دستورها	در کار روزانه
DDL	ساختن و تغییر ساختار	CREATE · ALTER · DROP	ساختن جدول جدید برای یک گزارش
DML	اضافه، ویرایش و حذف داده	INSERT · UPDATE · DELETE	لود داده‌ی دیروز در پایپ‌لاین
DQL	خوندن داده	SELECT	تقریباً هر سوالی از داده
DCL	کی به چی دسترسی داره	GRANT · REVOKE	دسترسی فقط‌خواندنی برای تحلیل‌گر
TCL	چند دستور با هم: یا همه یا هیچ	BEGIN · COMMIT · ROLLBACK	ثبت سفارش و پرداختش با هم

DDL در عمل: ساختن و تغییر جدول

CREATE TABLE

یک جدول تازه

اسم ستون‌ها، نوع داده و قانون‌ها رو همین‌جا تعریف می‌کنیم.

```
CREATE TABLE products (
  product_id  INT PRIMARY KEY,
  name        TEXT NOT NULL,
  unit_price  NUMERIC(10,2)
);
```

ALTER TABLE

تغییر جدولی که هست

ستون اضافه کن، اسم عوض کن؛ داده‌ها سر جاشون می‌مونن.

```
ALTER TABLE products
  ADD COLUMN is_active BOOLEAN DEFAULT true;

ALTER TABLE products
  RENAME COLUMN name TO product_name;
```

یادتون باشه: DDL ساختار رو عوض می‌کنه، نه داده رو. روی جدولی که برنامه‌ها ازش استفاده می‌کنن، با هماهنگی و احتیاط.

قانون‌هایی که دیتابیس خودش چک می‌کنه

PRIMARY KEY

شناسه‌ی یکتا

هر ردیف یک شناسه‌ی تکرارنشدنی و غیرخالی داره.

```
customer_id INT PRIMARY KEY
```

FOREIGN KEY

ارجاع معتبر

سفارش برای مشتری‌ای که وجود نداره ثبت نمی‌شه.

```
REFERENCES customers (customer_id)
```

NOT NULL · UNIQUE

اجباری و بدون تکرار

مثلاً ایمیل نباید خالی یا تکراری باشه.

```
email TEXT NOT NULL UNIQUE
```

CHECK

هر شرط دلخواه

مثلاً قیمت محصول همیشه باید مثبت باشه.

```
CHECK (unit_price > 0)
```

DML در عمل: اضافه، ویرایش، حذف

INSERT

اضافه کردن

یک ردیف تازه به جدول اضافه کن.

```
INSERT INTO products
  (product_id, product_name,
   category, unit_price)
VALUES (61, 'Laptop Stand',
        'Electronics', 39.90);
```

UPDATE

ویرایش

ستونی از ردیف‌هایی که شرط دارن رو عوض کن.

```
UPDATE products
SET unit_price = 35.00
WHERE product_id = 61;
```

DELETE

حذف

ردیف‌هایی که شرط دارن رو پاک کن.

```
DELETE FROM products
WHERE product_id = 61;
```

خطرناک‌ترین اشتباه: UPDATE و DELETE بدون WHERE روی کل جدول اجرا می‌شن. توی بخش تله‌ها می‌بینیم چطور ازش در امان بمونیم.

سه راه برای پاک کردن، سه نتیجه‌ی متفاوت

	DELETE	TRUNCATE	DROP TABLE
چی پاک می‌شه؟	ردیف‌ها؛ همه یا با شرط	همه‌ی ردیف‌ها	کل جدول، با ساختارش
شرط WHERE	داره	نداره	نداره
جدول بعدش	می‌مونه	می‌مونه، خالی	دیگه وجود نداره
روی جدول بزرگ	کُند؛ ردیف به ردیف	خیلی سریع	خیلی سریع

بعد از COMMIT هیچ‌کدوم برنمی‌گردن؛ فقط بکاپ نجاتتون می‌ده.

TCL و DCL در عمل: تراکنش و دسترسی

TCL · TRANSACTION

یا همه، یا هیچ

سفارش و پرداختش با هم ثبت بشن؛ اگه یکی خطا داد، هیچ‌کدوم.

```
BEGIN;
INSERT INTO orders   (...) VALUES (...);
INSERT INTO payments (...) VALUES (...);
COMMIT;   -- on error: ROLLBACK
```

DCL · ACCESS

هر کس فقط چیزی که لازم داره

تحلیل‌گر یا ایجنت AI فقط بخونه، و جدول پرداخت‌ها رو نبینه.

```
CREATE ROLE analyst LOGIN PASSWORD '...';
GRANT SELECT ON ALL TABLES
  IN SCHEMA public TO analyst;
REVOKE SELECT ON payments FROM analyst;
```

یادآوری دردسرهای فایل: «خرابی وسط کار» رو تراکنش حل می‌کنه و «دسترسی» رو GRANT. دیتابیس دقیقاً برای همین‌ها ساخته شده.

بخش ۰۳

خواندن داده

از یک SELECT ساده تا خلاصه کردن داده با GROUP BY.

اولین کوئری

```sql
SELECT full_name, city
FROM customers
WHERE country = 'Iran'
ORDER BY full_name
LIMIT 5;
```

SELECT

کدوم ستون‌ها رو می‌خوای ببینی؟ * یعنی همه.

FROM

از کدوم جدول بخونه؟

WHERE

کدوم ردیف‌ها؟ فقط اونایی که شرط براشون درسته.

ORDER BY · LIMIT

به چه ترتیبی، و چندتا؟ اینجا: بر اساس اسم، فقط ۵ تای اول.

جعبه‌ابزار WHERE

عملگر	یعنی	مثال
= <> < >	مقایسه	order_amount > 500
AND · OR · NOT	ترکیب چند شرط	status = 'paid' AND order_amount > 500
IN	یکی از چند مقدار	city IN ('Tehran', 'Shiraz')
BETWEEN	بین دو مقدار، با خود دو سر	unit_price BETWEEN 10 AND 50
LIKE	شبیه یک الگو؛ % یعنی هر چیزی	email LIKE 'sara%'
IS NULL	مقدار نداره	city IS NULL

از ردیف به خلاصه

```sql
SELECT country,
       COUNT(*) AS customers
FROM customers
GROUP BY country
HAVING COUNT(*) > 400
ORDER BY customers DESC;
```

COUNT · SUM · AVG · MIN · MAX

تابع‌های تجمیعی؛ چند ردیف رو به یک عدد تبدیل می‌کنن.

GROUP BY

گروه‌بندی؛ به ازای هر کشور یک ردیف خروجی.

WHERE vs HAVING

WHERE ردیف‌ها رو قبل از گروه‌بندی فیلتر می‌کنه، HAVING گروه‌ها رو بعدش.

DISTINCT

بدون تکرار؛ SELECT DISTINCT city یعنی فهرست شهرها، هر کدوم یک بار.

SQL به این ترتیب اجرا می‌شه



WHERE قبل از SELECT اجرا می‌شه؛ پس هنوز اسم net رو نمی‌شناسه و این کوئری خطا می‌ده:

```
SELECT order_amount * 0.9 AS net FROM orders WHERE net > 100;
```

راه درست: همون عبارت رو در WHERE تکرار کن، یا اول با CTE بسازش.

شرط و NULL داخل SELECT

CASE WHEN

if و else داخل SQL

از روی شرط‌ها یک ستون تازه می‌سازه.

```
CASE WHEN unit_price < 20  THEN 'cheap'
     WHEN unit_price < 100 THEN 'mid'
     ELSE 'premium'
END AS price_band
```

COALESCE · NULLIF

کنار اومدن با NULL

COALESCE اولین مقدار غیرخالی رو می‌ده. NULLIF اگه دو مقدار برابر باشن NULL می‌ده.

```
COALESCE(country, 'unknown')

total / NULLIF(order_count, 0)
```

یادتون باشه: NULL با هیچ‌چیز برابر نیست، حتی با خودش. برای همین به‌جای = NULL باید IS NULL نوشت.

بخش ۰۴

ترکیب جدول‌ها

داده رو تیکه‌تیکه ذخیره کردیم؛ حالا دوباره کنار هم می‌ذاریمش.

انواع JOIN

INNER JOIN

فقط جفت‌ها

ردیف‌هایی که در هر دو جدول جفت دارن.

مشتری‌هایی که سفارش دارن

LEFT JOIN

همه‌ی جدول اول

همه‌ی ردیف‌های جدول اول؛ اگه جفت نداشت، ستون‌های جدول دوم NULL می‌شن.

همه‌ی مشتری‌ها، با سفارش یا بی‌سفارش

RIGHT JOIN

همه‌ی جدول دوم

برعکس LEFT. معمولاً جای جدول‌ها رو عوض می‌کنن و LEFT می‌نویسن.

کم‌کاربرد

FULL JOIN

همه از هر دو طرف

هر ردیفی از هر دو جدول، چه جفت داشته باشه چه نه.

برای مقایسه‌ی دو منبع داده

JOIN در عمل

```sql
SELECT c.full_name,
       o.order_id,
       o.order_amount
FROM customers AS c
LEFT JOIN orders AS o
  ON o.customer_id = c.customer_id;
```

ON

شرط وصل: کدوم ستون‌ها باید با هم برابر باشن.

AS

c و o اسم کوتاه جدول‌ها هستن؛ کوئری خواناتر می‌شه.

NULL

مشتری بی‌سفارش با order_id خالی میاد. ۷۵۰ مشتری این‌طوری‌ان.

ROWS

مشتری‌ای که ۵ سفارش داره، ۵ ردیف می‌شه.

نکتهٔ کلیدی

قبل از هر JOIN بپرس:
هر ردیفِ خروجی یعنی چی؟

یک مشتری؟ یک سفارش؟ یک قلم کالا؟ به این می‌گن «گرین» (grain). بیشتر جواب‌های غلط از همین‌جا شروع می‌شن.

ابزارهای دیگه برای ترکیب

UNION ALL · UNION

زیر هم گذاشتن

دو نتیجه با ستون‌های یکسان رو زیر هم می‌ذاره. UNION تکراری‌ها رو هم حذف می‌کنه و کندتره.

```
SELECT ... UNION ALL SELECT ...
```

EXISTS · NOT EXISTS

هست یا نیست؟

مثلاً مشتری‌هایی که هیچ‌وقت سفارش ندادن.

```
WHERE NOT EXISTS (SELECT 1 FROM ...)
```

SUBQUERY

کوئری داخل کوئری

نتیجه‌ی یک کوئری، ورودی کوئری دیگه.

```
WHERE x > (SELECT AVG(x) FROM ...)
```

CTE

قدم‌به‌قدم نوشتن

با WITH هر مرحله یک اسم می‌گیره؛ خوندن و دیباگش خیلی راحت‌تره.

```
WITH paid AS (...) SELECT ... FROM paid
```

بخش ۰۵

حلقه نزن!

SQL با مجموعه‌ها کار می‌کنه، نه ردیف به ردیف.

به‌جای حلقه، یک کوئری

N+1 · SLOW

یک کوئری برای هر مشتری

۵۰۰۰ مشتری یعنی ۵۰۰۰ بار رفت‌وبرگشت به دیتابیس.

```
for cid in customer_ids:
    cur.execute(
        "SELECT SUM(order_amount) "
        "FROM orders "
        "WHERE customer_id = %s", (cid,))
```

SET-BASED · FAST

یک کوئری برای همه

دیتابیس کل کار رو یک‌جا و بهینه انجام می‌ده.

```
SELECT customer_id,
       SUM(order_amount)
FROM orders
GROUP BY customer_id;
```

همین منطق برای UPDATE هم هست. به‌جای حلقه‌ای که تک‌تک ردیف‌ها رو آپدیت کنه، یک UPDATE با شرط بنویسید؛ دیتابیس برای کار روی مجموعه ساخته شده.

Window Function: حساب کن، ولی ردیف‌ها رو نگه دار

ROW_NUMBER

شماره‌گذاری

سفارش‌های هر مشتری رو به ترتیب تاریخ شماره بزن.

```
ROW_NUMBER() OVER (
  PARTITION BY customer_id
  ORDER BY order_at)
```

LAG

مقدار قبلی

مبلغ سفارش قبلی همین مشتری، کنار سفارش فعلی.

```
LAG(order_amount) OVER (
  PARTITION BY customer_id
  ORDER BY order_at)
```

SUM() OVER

جمع تا اینجا

مجموع خرید مشتری تا همین سفارش.

```
SUM(order_amount) OVER (
  PARTITION BY customer_id
  ORDER BY order_at)
```

فرقش با GROUP BY چیه؟ GROUP BY ردیف‌ها رو یکی می‌کنه. Window Function کنار هر ردیف یک عدد حساب می‌کنه و ردیف‌ها سر جاشون می‌مونن. PARTITION BY یعنی «برای هر مشتری جدا».

بخش ۰۶

شش تله‌ای که جواب غلط می‌دن

کوئری خطا نمی‌ده، ولی عدد غلطه؛ خطرناک‌ترین نوع باگ.

تله ۱: Fan-out، وقتی JOIN عددها رو چند برابر می‌کنه

WRONG

سه جدول رو مستقیم JOIN کن

سفارش سه‌قلمی با دو قسط، شش بار شمرده می‌شه.

```
SELECT SUM(o.order_amount)
FROM orders o
JOIN order_items i USING (order_id)
JOIN payments p    USING (order_id)
WHERE o.status = 'paid';
-- 18,366,226
```

RIGHT

اول خلاصه کن، بعد JOIN

پرداخت‌ها رو اول تا گرین سفارش جمع بزن.

```
WITH paid AS (SELECT order_id, SUM(amount) AS paid
              FROM payments GROUP BY order_id)
SELECT SUM(o.order_amount)
FROM orders o JOIN paid p USING (order_id)
WHERE o.status = 'paid';
-- 5,770,297
```

چرا؟ هر سفارش در «تعداد قلم × تعداد قسط» ضرب می‌شه. چک سریع: بعد از هر JOIN، تعداد ردیف‌ها رو با تعداد سفارش‌های یکتا مقایسه کن؛ اگه برابر نیستن، fan-out داری.

تله ۲: LEFT JOIN که بی‌صدا INNER می‌شه

WRONG

شرط جدول دوم در WHERE

مشتری‌هایی که سفارش paid ندارن، حذف می‌شن.

```
SELECT c.customer_id, COUNT(o.order_id)
FROM customers c
LEFT JOIN orders o ON o.customer_id = c.customer_id
WHERE o.status = 'paid'
GROUP BY c.customer_id;
-- 4,250 rows
```

RIGHT

شرط رو ببر توی ON

همه‌ی مشتری‌ها می‌مونن؛ بی‌سفارش‌ها با صفر.

```
SELECT c.customer_id, COUNT(o.order_id)
FROM customers c
LEFT JOIN orders o ON o.customer_id = c.customer_id
                  AND o.status = 'paid'
GROUP BY c.customer_id;
-- 5,000 rows
```

چرا؟ برای مشتری بی‌سفارش، ستون status خالی (NULL) میاد و شرط WHERE براش درست نمی‌شه؛ پس اون ردیف حذف می‌شه و LEFT JOIN عملاً INNER می‌شه.

تله ۳: NULL یعنی «نمی‌دونیم»، نه «خالی»

WRONG

مقایسه‌ی معمولی با NULL

مشتری‌هایی که کشورشون نامعلومه، جا می‌مونن.

```sql
SELECT COUNT(*) FROM customers
WHERE country <> 'Iran';     -- 1,473

SELECT COUNT(*) FROM customers
WHERE city = NULL;           -- 0
```

RIGHT

صریح با NULL برخورد کن

NULL رو جدا در نظر بگیر.

```sql
SELECT COUNT(*) FROM customers
WHERE country IS DISTINCT FROM 'Iran';  -- 1,621

SELECT COUNT(*) FROM customers
WHERE city IS NULL;          -- 409
```

چرا؟ هر مقایسه با NULL جوابش «نامعلوم»ه و WHERE فقط ردیف‌های «درست» رو نگه می‌داره. حواستون باشه COUNT(city) و AVG هم NULLها رو نادیده می‌گیرن.

تله ۴: NOT IN و یک NULL که همه‌چیز رو صفر می‌کنه

WRONG

NOT IN با زیرکوئری

شهرهایی که هیچ مشتری ترکیه‌ای اونجا نیست.

```
SELECT COUNT(*) FROM customers
WHERE city NOT IN (
  SELECT city FROM customers
  WHERE country = 'Turkey');
-- 0
```

RIGHT

NOT EXISTS

با NULL درست رفتار می‌کنه.

```
SELECT COUNT(*) FROM customers c
WHERE NOT EXISTS (
  SELECT 1 FROM customers t
  WHERE t.country = 'Turkey'
    AND t.city = c.city);
-- 4,543
```

چرا؟ ۴۱ مشتری ترکیه شهر ندارن، پس توی لیست NULL هست. دیتابیس نمی‌تونه مطمئن بگه شهری «توی لیست نیست»، پس هیچ ردیفی برنمی‌گردونه.

تله ۵: بازه‌ی زمانی؛ روز آخر کجا رفت؟

WRONG

BETWEEN با تاریخ

تاریخ بدون ساعت یعنی نیمه‌شبِ شروع اون روز.

```sql
SELECT SUM(order_amount)
FROM orders
WHERE order_at BETWEEN '2026-09-01'
                   AND '2026-09-30';
-- 849,316.84
```

RIGHT

بازه‌ی نیمه‌باز

از شروع، تا قبل از روز بعد.

```sql
SELECT SUM(order_amount)
FROM orders
WHERE order_at ⩾ '2026-09-01'
  AND order_at <  '2026-10-01';
-- 876,572.65
```

حتی ساعت ۲۳:۵۹:۵۹ هم کافی نیست: سفارش ۷۷۷ نیم ثانیه بعدش ثبت شده و جا می‌مونه. همیشه بزرگ‌تر-مساوی شروع، و کوچک‌تر از پایان.

تله ۶: UPDATE و DELETE؛ اول ببین، بعد تغییر بده

WRONG

یک WHERE جا افتاده

همه‌ی ۵۰ هزار سفارش لغو می‌شن.

```
UPDATE orders
SET status = 'cancelled';
-- UPDATE 50000
```

RIGHT

اول SELECT، بعد داخل تراکنش

تعداد رو ببین، اگه درست بود COMMIT کن.

```
BEGIN;
SELECT COUNT(*) FROM orders
WHERE status = 'pending'
  AND order_at < '2025-02-01';   -- 47
UPDATE orders SET status = 'cancelled' WHERE ...;
COMMIT;   -- or ROLLBACK
```

عادت خوب: WHERE رو اول با SELECT امتحان کن، بعد همون رو به UPDATE بده. بعد از COMMIT دیگه برگشتی نیست؛ فقط بکاپ.

قبل از تحویل هر کوئری

- هر ردیف خروجی یعنی چی؟ (گرین)
- بعد از JOIN، تعداد ردیف‌ها با تعداد کلیدهای یکتا می‌خونه؟
- NULLها کجا رفتن؟ حذف شدن یا جایی شمرده نشدن؟
- بازه‌های زمانی نیمه‌بازن؟ (>= شروع و < پایان)
- عدد نهایی رو با یک منبع دیگه یا یک حساب سرانگشتی چک کردم؟

تا جلسهٔ بعد

HOMEWORK

۱۲ تمرین

هر چیزی که امروز دیدیم، روی همین دیتاست. جواب‌ها کنارش هست، ولی اول خودتون حل کنید.

exercises/part1.md

READ

W3Schools

از SQL Intro تا SQL Joins رو بخونید، به‌اضافه‌ی Group By، Having، Case، Exists و Null Functions. هر صفحه یک ویرایشگر داره که همون‌جا امتحان کنید.

w3schools.com/sql

NEXT

جلسهٔ بعد

تله‌های بیشتر، الگوهای کلاسیک مسائل، SQL در پایپ‌لاین داده و SQL in Scale.

SESSION 07 · PART 2

[END] $ exit 0

پایان جلسه

سوال؟

جلسهٔ بعد: SQL، قسمت دوم — الگوهای مسائل، SQL در پایپ‌لاین و SQL in Scale

عضویت در کانال تلگرام دوره

t.me/data_ai_engineering